

JavaScript — Anotação de 1 página: `forEach()`

Conceito principal

`forEach()` é um método usado para percorrer os elementos de um array, executando uma ação para cada elemento. A ideia é: “**para cada item do array, execute esta função**”.

```
const frutas = ["maçã", "banana", "laranja"];

frutas.forEach((fruta) => {
  console.log(fruta);
});
```

Como funciona?

O **`forEach()`** recebe uma **callback**, ou seja, uma função que será chamada para cada elemento do array. A arrow function é muito comum nesse caso.

```
frutas.forEach((fruta) => {
  // fruta = elemento atual
  console.log(fruta);
});
```

Parâmetros disponíveis

A callback pode receber até três informações: **elemento atual**, **índice** e **o próprio array**.

```
frutas.forEach((fruta, indice, array) => {
  console.log(indice, fruta);
});
```

`forEach()` x **`for...of`**

`forEach()` é ótimo quando queremos apenas executar uma ação para cada item. Já **`for...of`** é mais adequado quando precisamos controlar o fluxo, especialmente com **`await`**, **`break`** ou **`continue`**.

```
// forEach
usuarios.forEach((usuario) => {
  console.log(usuario);
});

// for...of – útil com async/await
for (const usuario of usuarios) {
  await cadastrarUsuario(usuario);
}
```

■ Atenção no Playwright

Evite depender de **`forEach(async (...) => ...)`** quando a ordem e a espera das operações forem importantes. O **`forEach()`** não espera as Promises da callback. Em automação, prefira frequentemente **`for...of + await`** para executar cada passo de forma previsível.

Exemplo em automação

```
const botoes = ["Salvar", "Cancelar", "Voltar"];

for (const botao of botoes) {
  await page.getByRole("button", { name: botao }).click();
}
```

■ Para memorizar

Array → **`forEach()`** → **callback** → **cada elemento** → **executa uma ação**.

Regra prática: use **`forEach()`** para percorrer e executar ações simples; use **`for...of`** quando precisar controlar o fluxo ou trabalhar com **`async/await`**.